

Intraprocedural Dataflow Analysis for Software Product Lines

Claus Brabrand^{1,2} Márcio Ribeiro^{2,3} Társis Tolêdo² Johnni Winther⁴ Paulo Borba²

¹ IT University of Copenhagen, Rued Langgaards Vej 7, DK-2300, Copenhagen, Denmark

² Federal University of Pernambuco, Av. Prof. Luis Freire, 50.740-540, Recife, Brazil

³ Federal University of Alagoas, Av. Lourival de Melo Mota, 57.072-970, Maceió, Brazil

⁴ Aarhus University, Nordre Ringgade 1, DK-8000, Aarhus, Denmark

{brabrand@itu.dk, {mmr3, twt, phmb}@cin.ufpe.br, jw@cs.au.dk}

Abstract

Software product lines (SPLs) are commonly developed using annotative approaches such as conditional compilation that come with an inherent risk of constructing erroneous products. For this reason, it is essential to be able to analyze SPLs. However, as dataflow analysis techniques are not able to deal with SPLs, developers must generate and analyze all valid methods individually, which is expensive for non-trivial SPLs. In this paper, we demonstrate how to take *any* standard intraprocedural dataflow analysis and *automatically* turn it into a *feature-sensitive* dataflow analysis in four different ways. All are capable of analyzing *all* valid methods of an SPL without having to generate all of them explicitly. We have implemented all analyses using Soot's intraprocedural dataflow analysis framework and experimentally evaluated their performance and memory characteristics on four qualitatively different SPLs. Further, we show how empirical observations lead to heuristics on how to intraprocedurally combine the previous analyses. From this we are able to construct an analysis strategy that is faster than all of the individual analyses on all of the benchmarks. On our benchmarks, this strategy is on average six times faster than the brute force approach on our SPLs.

Categories and Subject Descriptors D.2.4 [Software Engineering]: Software/Program Verification; F.3.2 [Theory of Computation]: Semantics of Programming Languages — Program Analysis

General Terms Performance, Experimentation, Design

Keywords Dataflow Analysis, Software Product Lines.

1. Introduction

A software product line (SPL) is a set of software products that share common functionality and are generated from reusable assets. These assets specify common and variant behavior targeted at a specific set of products, usually bringing productivity and time-to-market improvements [11, 28]. Developers often implement variant behavior and associated features with conditional compilation constructs like `#ifdef` [2, 20], mixing common, optional, and even alternative and conflicting behavior in the same code asset. In these cases, assets are not valid programs or program elements in the underlying language. We can, however, use assets to generate valid programs by evaluating the conditional compilation constructs using preprocessing tools.

Since code assets might not be valid programs or program elements, existing standard dataflow analyses, which are for instance essential for supporting optimization [22] and maintenance [31] tasks, cannot be directly used to analyze code assets. To analyze an SPL using intraprocedural analysis, developers then have to generate all possible methods and separately analyze each one with conventional single-program dataflow analyses. In this case, generating and analyzing each method can be expensive for non-trivial SPLs. Consequently, interactive tools for single-program development might not be usable for SPL development because they rely on fast dataflow analyses and have to be able to quickly respond when the programmer performs tasks such as code refactoring [14]. Also, this is bad for maintenance tools [31] that help developers understand and manage dependencies between features.

To solve this problem and enable more efficient dataflow analysis of SPLs, we propose four approaches for taking any standard dataflow analysis and automatically lifting it into a corresponding *feature-sensitive* analysis that we can use to directly analyze code assets. The approaches analyze all configurations and thus avoid explicitly generating all possible methods of an SPL. Although we focus on SPLs developed

with conditional compilation constructs, our results apply to other similar annotative variability mechanisms [20].

We evaluate our four *feature-sensitive* approaches (*consecutive*, *simultaneous*, *simultaneous with sharing*, and *simultaneous with sharing and merging*) and compare them with a *brute force* intraprocedural approach that generates and analyzes all possible methods individually. We report on a number of performance and memory consumption experiments using the ubiquitous dataflow analysis, *reaching definitions* [27], and four SPLs from different domains, with qualitatively different numbers of features, products, `#ifdef` statements, and other factors that might impact performance and memory usage results.

From experimental data we derive an analysis strategy which intraprocedurally combines our above analyses to heuristically select a good analysis for a given method. This combined strategy is faster than all individual analyses on all benchmarks. On average over the benchmarks, this feature-sensitive strategy is six times faster than the feature-oblivious brute-force analysis.

List of Contributions

Our paper presents the following contributions:

- four qualitatively different ways of automatically deriving *feature-sensitive* analyses capable of analyzing all configurations of a SPL (Section 5) from conventional *feature-oblivious* dataflow analyses; in particular:
 - a *consecutive* approach that analyzes all configurations, one at a time;
 - a *simultaneous* approach that analyzes all configurations at the same time;
 - a *simultaneous with sharing* approach that in addition to the simultaneous approach is capable of sharing information among the configurations; and
 - a *simultaneous with sharing and merging* approach that in addition to the previous analysis is also capable of merging equivalent configurations during analysis.
- *experimental validation* of our ideas using the ubiquitous reaching definitions dataflow analysis on four SPL benchmarks with and without taking the feature model into consideration; and
- an *intraprocedurally combined strategy*, derived from experimental data, capable of combining the previous analyses to, on average, analyze the benchmarks six times faster than the brute-force analysis.

Organization of the Paper

The paper is organized as follows. Using concrete examples, Section 2 discusses and motivates the need for dataflow analysis of software product lines. Then, we introduce conditional compilation and feature models. After that, we briefly

recall basic dataflow analysis concepts and present the main contributions of this paper.

2. Motivating Examples

To better illustrate the issues we are addressing in this paper, we now present a motivating example based on the Lampiro SPL¹. Lampiro is an instant-messaging client developed in Java ME and its features are implemented using `#ifdefs`.

Figure 1(a) shows the essence of a code snippet extracted from Lampiro implemented in Java with the Antenna² pre-processor. As can be seen, if the GLIDER feature is *not* enabled (see the `#ifndef` statement), the `logo` variable receives an image instantiated by the `createImage` method, so it is *initialized*. However, this variable is *uninitialized* if the GLIDER feature *is* enabled in the product.

Figure 1(b) illustrates another common error in an SPL. Here, the `display` variable is assigned a new `VideoDisplay` object whenever the feature VIDEO is enabled. However, if this feature is disabled, the last line of the example program will produce a *null-pointer exception* when the `init` method is invoked on `display` (which is `null`).

Both examples so far, have been errors inherent to the Java language. Inappropriate usage of libraries can also cause SPL errors. Also, errors are not necessarily as simple as suggested by the previous examples, limited to whether or not a single `#ifdef` block is enabled or disabled. Errors may also depend intricately on combinations of `#ifdef` blocks. The code fragment in Figure 1(c) sketches a simple combination case that appear to contain an erroneous “double closing” of an output resource (the resource is closed in both `#ifdefs`). However, it might be the case that the HIGH and LOW features are so-called *mutually exclusive features*; i.e., HIGH is enabled if and only if LOW is disabled. The program will then, in fact, not have the double closing error. In order to analyze SPL programs we thus need to know which combinations of features are designated as valid. We return to this point later.

Figure 1(d) shows yet another kind of error (inspired by [8]) where unintended information flow through the program compromise sensitive information. In this contrived example, the password is disclosed in products where the features OOPS and (YIKES or GOSH) are enabled.

The above examples illustrate possible errors when developing SPLs using conditional compilation. Despite their widespread usage to implement variability in SPLs, `#ifdefs` pollute the code, lack separation of concerns, and make maintenance tasks harder [12, 24, 25, 33].

For maintaining SPLs, it is important to be able to analyze their code so as to issue error messages whenever developers are introducing errors into the SPL before compilation of the code of an actual product. Detecting errors at *SPL development time* rather than *product compilation time* which

¹<http://lampiro.blundo.com/>

²<http://antenna.sourceforge.net/>

```

Image logo;
...
#ifdef GLIDER
... logo = Image.createImage("icon.png"); ...
#endif
...
UILabel uimg = new UILabel(logo);

```

(a) Uninitialized variable when GLIDER is enabled.

```

Display display = null;
...
#ifdef VIDEO
... display = new VideoDisplay(1024, 768); ...
#endif
...
display.init();

```

(b) Null-pointer exception when VIDEO is disabled.

```

Outputter out = new Outputter();
out.open();
...
#ifdef HIGH
... out.close(); ...
#endif
...
#ifdef LOW
... out.close(); ...
#endif

```

(c) Double close when HIGH and LOW are enabled.

```

String s, t;
String password = "ToP-sEcReT";
...
#ifdef OOPS
... s = password; ...
#endif
...
#ifdef YIKES or GOSH
... t = "msg: " + s; ...
#endif
...
out.print(t);

```

(d) Password disclosed when OOPS and (YIKES or GOSH) are enabled.

Figure 1. Four example SPL program fragments with qualitatively different kinds of SPL errors.

occurs after *SPL instantiation time* can make a substantial difference in whether or not errors are detected in an SPL and whether or not the original programmer is at all present to correct the error. All errors in Figure 1 can be caught by conventional dataflow analyses analyzing for *uninitialized variables*, *null-pointers*, *object state*, and *information flow*. However, such conventional analyses need to be “lifted” to analyze not a single program, but a *family of programs* expressed as a software product line.

Another analysis is of particular interest to us; namely the classical *reaching definitions* analysis. This analysis is commonly used to produce a *definition-usage* (aka., *def-use*) graph which may in turn be used to compute data (variable assignment) dependencies among different parts of a program. This analysis is particularly interesting for SPLs in cases where multiple developers collaborate on programming an SPL with each developer responsible for his own set of features. In such cases, one developer changing the value of a variable belonging to a feature he or she is maintaining might inadvertently influence or break other features maintained by another programmer. The reaching definitions analysis could then be used to warn the first programmer of the other features his change may affect. We have proposed the idea of providing information about this kind of feature dependency in an SPL [31]. This was our original motivation for adapting dataflow analysis for SPLs. In this paper, we will use the *reaching definition analysis* as our analysis benchmark.

To intercept dependencies and errors like the ones we have seen above, we need dataflow analyses to work on sets of SPL assets, like the ones using conditional compilation. However, programmers must resort to generating all possible methods and separately analyzing each one by using the conventional single-program dataflow analysis. Depending on the size of the SPL, this can be costly, which may be a problem for interactive tools that analyze SPL code, for example. As we shall see in Section 7, we are able to decrease such costs.

3. Conditional Compilation

In this section, we briefly introduce the `#ifdef` construction and *feature models*. We use a simplified `ifdef` construction the syntax of which is:

$$\begin{aligned}
 S &::= \text{"ifdef" "(" } \phi \text{ ")" } S \\
 \phi &::= f \in \mathbb{F} \mid \neg \phi \mid \phi \wedge \phi
 \end{aligned}$$

where S is a *Java Statement* and ϕ is a *propositional logic formula* over *feature names* where f is drawn from a finite alphabet of feature names, $\mathbb{F}$. Throughout the paper, however, we will use formulae from the full propositional logic extended with *true*, *false*, *disjunction*, *implication*, and *bi-implication* via syntactic sugar in the usual way.

In general, it is possible to make un-syntactic undisciplined `#ifdef` annotations (e.g., encapsulating *half* of a statement in an `#ifdef`). However, such vulgar constructions are fortunately usually easy to refactor into disciplined `#ifdefs` [18] respecting the syntactic structure of the un-

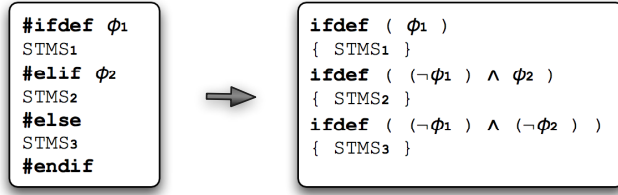


Figure 2. Example of normalization of `#ifdef` statements.

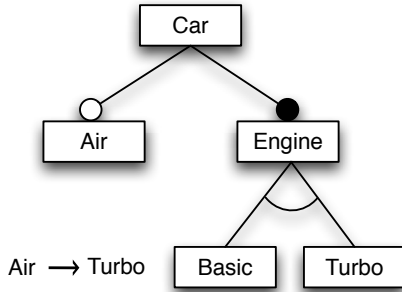


Figure 3. Diagrammatic representation of a feature model for a car.

derlying programming language. We further refactor these disciplined `#ifdefs` to eliminate any optional `#elif` and `#else` branches they might have by turning them into the normalized syntactic `ifdef` form listed in the BNF above. Figure 2 exemplifies this normalization process.

A *configuration*, $c \subseteq \mathbb{F}$, is a set of *enabled features*. A propositional logic formula, ϕ , gives rise to the *set of configurations*, $\llbracket \phi \rrbracket \subseteq 2^{\mathbb{F}}$, for which the formula is satisfied. For instance, given $\mathbb{F} = \{A, B, C\}$, the formula, $\phi = A \wedge (B \vee C)$ corresponds to the following set of configurations:

$$\llbracket A \wedge (B \vee C) \rrbracket = \{\{A, B\}, \{A, C\}, \{A, B, C\}\} \subseteq 2^{\mathbb{F}}$$

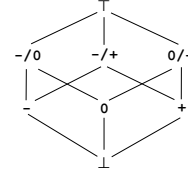
Feature Model

To yield only *valid* configurations, sets of configurations are usually further restricted by a so-called *feature model* often represented diagrammatically [17] as in Figure 3. Conceptually, however, when disregarding its structure, focussing only on validity of configurations, a feature model is just a propositional logic formula. Here is the feature model of Figure 3 represented as a propositional formula over the alphabet $\mathbb{F} = \{\text{Car}, \text{Engine}, \text{Air}, \text{Basic}, \text{Turbo}\}$:

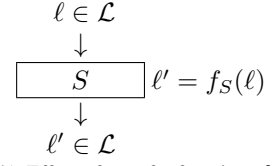
$$\psi_{\text{FM}} = \text{Car} \wedge \text{Engine} \wedge (\text{Basic} \leftrightarrow \neg \text{Turbo}) \wedge (\text{Air} \rightarrow \text{Turbo})$$

corresponding to the following set of *valid* configurations:

$$\llbracket \psi_{\text{FM}} \rrbracket = \{\{\text{Car}, \text{Engine}, \text{Basic}\}, \{\text{Car}, \text{Engine}, \text{Turbo}\}, \{\text{Car}, \text{Engine}, \text{Air}, \text{Turbo}\}\} \subseteq 2^{\mathbb{F}}$$



(a) Lattice for sign analysis.



(b) Effect of transfer function, f_S .

Figure 4. Lattice and transfer function.

Let $|\psi|$ denote the *number of configurations* in the interpretation of the formula ψ (i.e., $|\psi| =_{\text{def}} |\llbracket \psi \rrbracket|$). Notice that the number of *valid* configurations, $|\psi_{\text{FM}}|$, may be significantly smaller than the total number of configurations, $|2^{\mathbb{F}}|$. Indeed, in the above example we have that: $3 = |\psi_{\text{FM}}| < |2^{\mathbb{F}}| = 32$.

4. Dataflow Analysis

A Dataflow Analysis [22] is comprised of three constituents: 1) a *control-flow graph* (on which the analysis is performed); 2) a *lattice* (representing values of interest for the analysis); and 3) *transfer functions* (that simulate execution at compile-time). In the following, we briefly recall each of the constituents of the conventional (feature-oblivious) single-program dataflow analysis and how they may be combined to analyze an input program.

Control-Flow Graph: The *control-flow graph* (CFG) is the abstraction of an input program on which a dataflow analysis runs. A CFG is a directed graph where the nodes are the statements of the input program and the edges represent *flow of control* according to the semantics of the programming language. An analysis may be *intraprocedural* or *interprocedural*, depending on how functions are handled in the CFG. Here, we only consider intraprocedural analyses.

Lattice: The information calculated by a dataflow analysis is arranged in a *lattice*, $\mathcal{L} = (D, \sqsubseteq)$ where D is a set of elements and $\sqsubseteq$ is a *partial-order* on the elements [27]. Lattices are usually described diagrammatically using *Hasse Diagrams* which use the convention that $x \sqsubseteq y$ if and only if x is depicted *below* y in the diagram (according to the lines of the diagram). Figure 4(a) depicts such a diagram of a lattice for analyzing the sign of an integer. Each element of the lattice captures information of interest to the analysis; e.g., “+” represents the fact that a value is always *positive*, “0/+” that a value is always *zero-or-positive*. A lattice has two special elements; $\perp$ at the bottom of the lattice usually means “*not analyzed yet*” whereas $\top$ at the top of the lattice usually means “*analysis doesn’t know*”. The partial order induces a *least upper bound* operator, $\sqcup$, on the lattice elements [27] which is used to *combine* information during the analysis, when control-flows meet. For instance, $\perp \sqcup 0 = 0$, $0 \sqcup + = 0/+$, and $- \sqcup 0/+ = \top$.

Transfer Functions: Each statement, S , will have an associated *transfer function*, $f_S : \mathcal{L} \rightarrow \mathcal{L}$, which simulates the execution of S at compile-time (with respect to what is

being analyzed). Figure 4(b) illustrates the effect of executing transfer function f_S . Lattice element, ℓ , flows into the statement node, the transfer function computes $\ell' = f_S(\ell)$, and the result, ℓ' , flows out of the node. Here are the transfer functions for two assignment statements for analysing the sign of variable x using the sign lattice in Figure 4(a):

$$f_{x=0}(\ell) = 0 \quad f_{x++}(\ell) = \begin{cases} \top & \ell \in \{-/+, -/0, \top\} \\ + & \ell \in \{0, +, 0/+\} \\ -/0 & \ell = - \\ \perp & \ell = \perp \end{cases}$$

The transfer function, $f_{x=0}$, is the constant zero function capturing the fact that x will always have the value *zero* after execution of the statement $x=0$ irrespective of the original argument, ℓ , prior to execution. The transfer function, f_{x++} , simulates execution of $x++$; e.g., if x was *negative* ($\ell = -$) prior to execution, we know that its value after execution will always be *negative-or-zero* ($\ell' = -/0$). In order for a dataflow analysis to be well-defined, all transfer functions have to obey a *monotonicity* property [27].

Analysis: Figure 5 shows how to combine the control-flow graph, lattice, and transfer functions to perform dataflow analysis on a tiny example program. First (cf. Figure 5(a)), a control-flow graph is built from the program and annotated with *program points* (which are the *entry* and *exit* points of the statement nodes). In our example, there are four such program points which we label with the letters a to d . Second (cf. Figure 5(b)), the annotated CFG is turned into a *whole-program transfer function*, $T : \mathcal{L}^4 \rightarrow \mathcal{L}^4$, which works on four copies of the lattice, $\mathcal{L}$, since we have four program points (a to d). The entry point, a , is assigned an initialization value which depends on the analysis (here, $a = \perp$). For each program point, we simulate the effect of the program using transfer functions (e.g., $b = f_{x=0}(a)$) and the least-upper bound operator for combining flows (e.g., $c = b \sqcup d$). Third (cf. Figure 5(c)), we use the Fixed-Point Theorem [27] to compute the fixed-point of the function, T , by computing $T^i(\perp)$ for increasing values of i (depicted in the columns of the figure), until nothing changes. As seen in Figure 5(c), we reach the fixed-point in five iterations (since $T^4(\perp) = T^5(\perp)$) and the least-fixed point, and hence the result of the analysis, is: $a = \perp, b = 0, c = 0/+, d = +$ (which is the unique least fixed-point of T). From this we can deduce that the value of the variable x is always *zero* at program point b , it is *zero-or-positive* at point c , and *positive* at point d . (Note that, in practice, the fixed-point computation is often performed using more efficient iteration strategies.)

5. Dataflow Analyses for SPLs

In Section 2 we claimed that analyzing SPLs is important and that the naive brute force approach can be costly. In this section, we show how to take any feature-oblivious

```
void m() {
  int x = 0;
  ifdef (A) x++;
  ifdef (B) x--;
}
```

(a) Example SPL method

$c=\{A\} :$	$c=\{B\} :$	$c=\{A, B\} :$
int x=0;	int x=0;	int x=0;
x++;	x--;	x++;
		x--;

(b) and its three distinct method variants (configurations: $\{A\}$, $\{B\}$, and $\{A, B\}$).

Figure 6. A tiny example of an SPL method along with its three distinct method variants.

intraprocedural dataflow analysis and automatically turn it into a feature-sensitive analysis.

We present four different ways of performing intraprocedural dataflow analysis for software product lines (summarized in Figure 8). The four analyses calculate the same information, but in qualitatively different ways. To illustrate the principles, we use a deliberately simple example analysis; *sign analysis* of one variable, x , and use it to analyze an intentionally simple method, m (cf. Figure 6(a)) that increases and decreases a variable, depending on the features enabled.

The program uses features $\mathbb{F} = \{A, B\}$ and we assume it has a feature model $\psi_{\text{FM}} = A \vee B$ which translates into the following set of valid configurations: $\llbracket \psi_{\text{FM}} \rrbracket = \{\{A\}, \{B\}, \{A, B\}\}$.

A1: Brute Force Analysis (Feature Oblivious)

A software product line may be analyzed intraprocedurally by building *all* possible methods and analyzing them one by one using a conventional dataflow analysis as described in the previous section. A method with n features will give rise to 2^n possible end-product methods (minus those invalidated by the feature model). For our tiny example program that has two features, A and B , we have to build and analyze the *three* distinct methods as illustrated in Figure 6(b). Figure 7(a) depicts the result of analyzing each of these three products using the simple sign-of- x analysis.

A2: Consecutive Analysis

We can avoid explicitly building all methods individually by making a dataflow analysis *feature-sensitive*. Now, we show how to take any single-program dataflow analysis and automatically turn it into a feature-sensitive analysis, capable of analyzing all possible method variants. Firstly, we consider the consecutive analysis, named this way because we analyze each of the possible configurations one at a time. We render it feature-sensitive by instrumenting the CFG with sufficient information for the transfer functions to know whether a given statement is to be executed or not in each configuration.

Control-Flow Graph: For each node in the CFG, we associate the *set of configurations*, $\llbracket \phi \rrbracket$, for which the node's corresponding statement is executed. We refer to this process

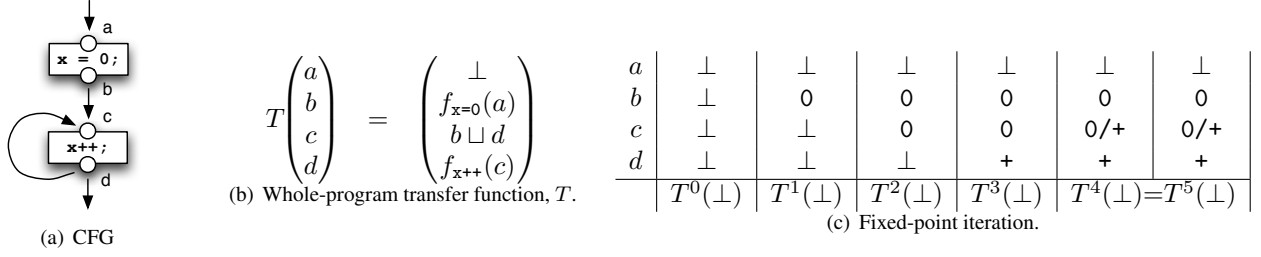


Figure 5. Combining CFG, lattice, and transfer functions to perform dataflow analysis (as a fixed-point iteration).

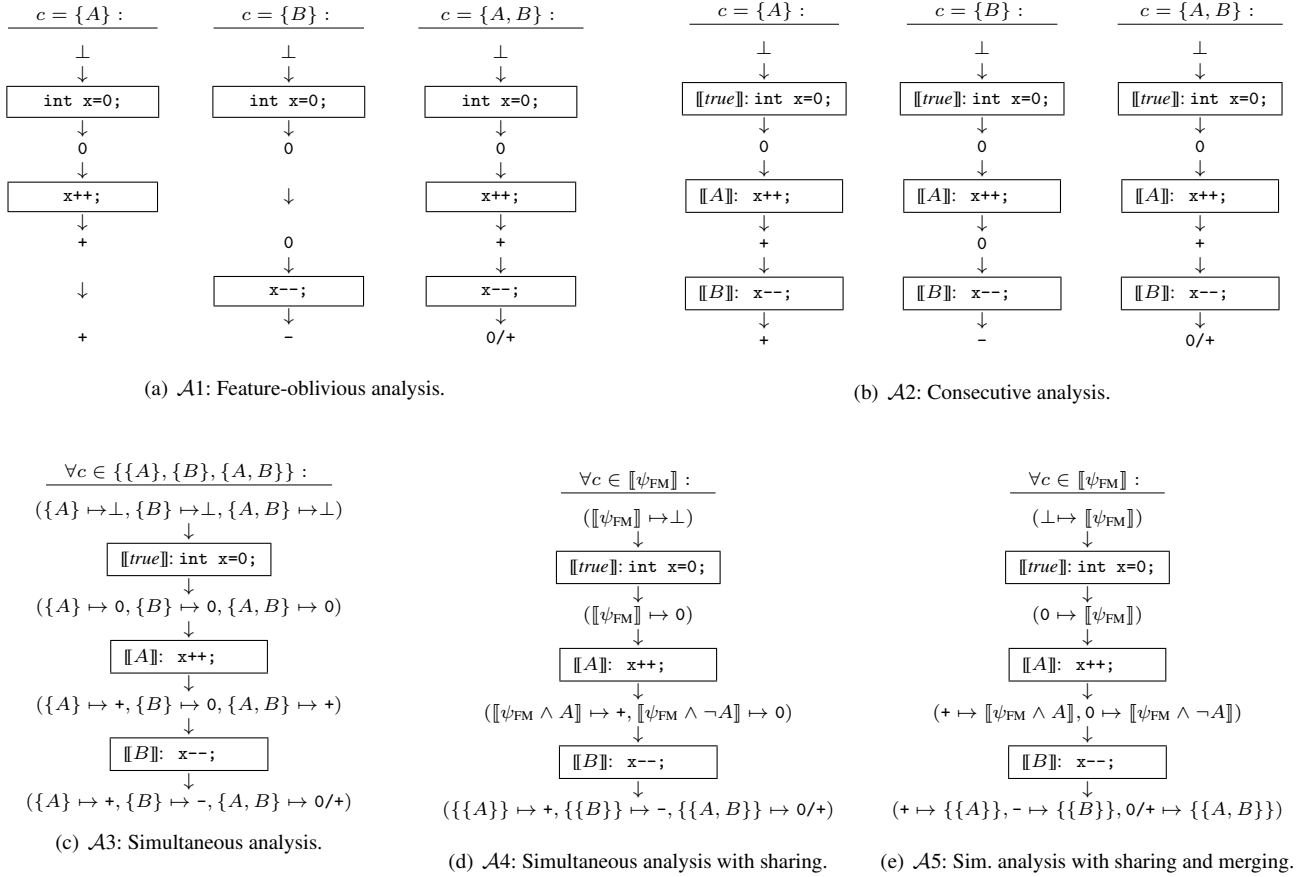
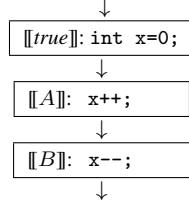


Figure 7. Results of using the four analyses on our tiny example method m (that increases and decreases variable, x).

as *CFG instrumentation*. Here is the instrumented CFG for our tiny method of Figure 6(a):



We label each node with “ $[[\phi]]: S$ ” where S is the statement and $[[\phi]]$ is the configuration set associated with the statement. Unconditionally executed statements (e.g., `int x=0;`) are associated with the set of all configurations, $[[true]]$. Statements that are nested inside several `ifdefs` will have the intersection of the configuration sets. For instance, statement S in “`ifdef ( $\phi_1$ ) ifdef ( $\phi_2$ )  $S$` ” will be associated with the set of configurations $[[\phi_1]] \cap [[\phi_2]] \equiv [[\phi_1 \wedge \phi_2]]$.

Lattice: Analyzing the configurations consecutively does not change the lattice, so the lattice of this feature-sensitive analysis is the same as that of the feature-oblivious analysis.

Transfer Functions: All we have to do in the feature-sensitive transfer function is use the associated configuration set, $[[\phi]]$, to figure out whether or not to execute the feature-oblivious transfer function, f_S , in a given configuration, c ; i.e., deciding $c \in [[\phi]]$ (cf. Figure 8). This membership test can be efficiently decided by using Binary Decision Diagrams [3] (BDDs) or *bit vectors* for representing feature formulae.

Since the lifting only either applies the feature-oblivious transfer function or copies the lattice value, the lifted transfer function is also always monotone.

Analysis: In order to analyze a program using $\mathcal{A}_2$, all we need to do is to combine the CFG, lattice, and transfer functions as explained in Section 4. Figure 7(b) shows the result of analyzing the increase-decrease method using this consecutive feature-sensitive analysis. As can be seen, the consecutive feature-sensitive analysis needs one fixed-point computation for each configuration. $\mathcal{A}_1$ and $\mathcal{A}_2$ compute the same information (the same fixed-point solution); the only difference is whether the applicability of statements, $c \in [[\phi]]$, is evaluated before or after compilation.

$\mathcal{A}_3$: Simultaneous Analysis

Another approach is to analyze all configurations *simultaneously* by using a *lifted* lattice that maintains one lattice element per valid configuration. As opposed to the consecutive analysis, the simultaneous analysis needs only *one* fixed-point computation. Again, this analysis will be *feature-sensitive* and it can also be automatically derived from the feature-oblivious analysis.

Control-Flow Graph: The CFG of $\mathcal{A}_3$ is the same as that of $\mathcal{A}_2$ as it already includes the necessary information

for deciding whether or not to simulate execution of a conditional statement.

Lattice: As explained, we lift the feature-oblivious lattice, $\mathcal{L}$, such that it has one element per valid configuration:

$$\mathcal{L}_3 = [[\psi_{FM}]] \rightarrow \mathcal{L}$$

Note that whenever $\mathcal{L}$ is a lattice, then so is $[[\psi_{FM}]] \rightarrow \mathcal{L}$ (which is isomorphic to $\mathcal{L}^{[[\psi_{FM}]]}$). An example element of this lattice is:

$$L_3 = (\{A\} \mapsto +, \{B\} \mapsto -, \{A, B\} \mapsto 0/+) \in [[\psi_{FM}]] \rightarrow \mathcal{L}$$

which corresponds to the information that: for configuration $\{A\}$, we know that the value of x is *positive* ($L_3(\{A\}) = +$); for $\{B\}$, we know x is *negative* ($L_3(\{B\}) = -$); and for $\{A, B\}$, we know it is *zero-or-positive* ($L_3(\{A, B\}) = 0/+$).

Transfer Functions: We *lift* the transfer functions correspondingly so they work on elements of the lifted lattice in a point-wise manner. The feature-oblivious transfer functions are applied only on the configurations for which the statement is executed. As an example, consider the statement “`ifdef (A) x++;`” where the effect of the lifted transfer function on the lattice element $L_3 = (\{A\} \mapsto 0, \{B\} \mapsto 0, \{A, B\} \mapsto 0)$ is:

$$\begin{array}{c}
 L_3 = (\{A\} \mapsto 0, \{B\} \mapsto 0, \{A, B\} \mapsto 0) \\
 \downarrow \\
 \boxed{[[A]]: x++;} \\
 \downarrow \\
 L'_3 = (\{A\} \mapsto +, \{B\} \mapsto 0, \{A, B\} \mapsto +)
 \end{array}$$

The transfer function of the feature-oblivious analysis is applied to each of the configurations for which the `ifdef` formula A is satisfied. Since $[[A]] = \{\{A\}, \{A, B\}\}$, this means that the function is applied to the lattice values of the configurations $\{A\}$ and $\{A, B\}$ with resulting value: $f_{x++}(0) = +$. The configuration $\{B\}$, on the other hand, does not satisfy the formula ($\{B\} \notin [[A]]$), so its value is left unchanged with value, $L'_3(\{B\}) = 0$. Figure 8 depicts and summarizes the effect of the lifted transfer function on the lifted lattice.

Since the feature-sensitive transfer function on $[[\psi_{FM}]] \rightarrow \mathcal{L}$ only applies monotone transfer functions on $\mathcal{L}$ in a point-wise manner, it is itself monotone. This guarantees the existence of a unique and computable solution.

Analysis: Again, we simply combine the lifted CFG, lifted lattice, and lifted transfer functions to achieve our feature-sensitive simultaneous configuration analysis. Figure 7(c) shows the result of analyzing the increase-decrease method using the simultaneous feature-sensitive analysis. From this we can read off the information about the sign of the variable x at different program points, for each of the valid configurations. For instance, at the end of the program in configuration $\{B\}$, we can see that x is always *negative*. Compared to $\mathcal{A}_2$, this analysis only has *one* fixed-point itera-

tion and thus potentially saves the overhead involved. However, it requires the maximum number of fixed-point iterations that are performed in any configuration of $\mathcal{A}2$ in order to reach its fixed-point because of the pointwise lifted lattice. Again, it is fairly obvious that $\mathcal{A}2$ and $\mathcal{A}3$ compute the same information; the only difference being that $\mathcal{A}2$ does one fixed-point iteration per valid configuration whereas $\mathcal{A}3$ computes the same information in one iteration in a pointwise manner.

$\mathcal{A}4$: Simultaneous Analysis with Sharing

Using the lifted lattice of the simultaneous analysis, it is possible to lazily share lattice values corresponding to configurations that are indistinguishable in the program being analyzed.

Control-Flow Graph: The CFG of $\mathcal{A}4$ is the same as that of $\mathcal{A}3$.

Lattice: To accomodate the sharing, the lifted lattice of $\mathcal{A}4$ will, instead of mapping configurations to base lattice values, map *sets of configurations* to base lattice values:

$$\mathcal{L}_4 = 2^{\llbracket \psi_{FM} \rrbracket} \hookrightarrow \mathcal{L}$$

This allows $\mathcal{A}4$ lattice values to *share* base lattice values for configurations that have not yet been distinguished by the analysis. For instance, the lifted lattice value of $\mathcal{A}3$, $L_3 = (\{A\} \mapsto \ell, \{B\} \mapsto \ell, \{A, B\} \mapsto \ell) \in \mathcal{L}_3$, can now be represented by $L_4 = (\llbracket A \vee B \rrbracket \mapsto \ell) \in \mathcal{L}_4$ where the three configurations, $\llbracket A \vee B \rrbracket = \{\{A\}, \{B\}, \{A, B\}\}$, *share* the base lattice value, ℓ .

Transfer Functions: The transfer functions of $\mathcal{A}4$ work by lazily *splitting* sets of configurations, $\llbracket \psi \rrbracket$, in two disjoint parts, depending on the feature constraint, ϕ , attached with the statement in question: A set of configurations for which the transfer function *should* be applied, $\llbracket \psi \wedge \phi \rrbracket$; and a set of configurations for which the transfer function should *not* be applied, $\llbracket \psi \wedge \neg \phi \rrbracket$; i.e.:

$$\begin{array}{c} L_4 = (\llbracket \psi \rrbracket \mapsto \ell, \dots) \\ \downarrow \\ \boxed{\llbracket \phi \rrbracket: S} \\ \downarrow \\ L'_4 = (\llbracket \psi \wedge \phi \rrbracket \mapsto f_S(\ell), \llbracket \psi \wedge \neg \phi \rrbracket \mapsto \ell, \dots) \end{array}$$

Note that $\llbracket \psi \wedge \phi \rrbracket \cup \llbracket \psi \wedge \neg \phi \rrbracket = \llbracket (\psi \wedge \phi) \vee (\psi \wedge \neg \phi) \rrbracket = \llbracket \psi \wedge (\phi \vee \neg \phi) \rrbracket = \llbracket \psi \wedge \text{true} \rrbracket = \llbracket \psi \rrbracket$ which means that $\llbracket \psi \rrbracket$ is split into two parts without losing any configurations. Of course, it might be the case that $\llbracket \psi \rrbracket$ is split into “nothing”, $\emptyset$, and “everything”, $\llbracket \psi \rrbracket$ (which happens whenever $\psi \wedge \phi \equiv \text{false}$ or $\psi \wedge \neg \phi \equiv \text{false}$). We eliminate such *false* constituents in order to ensure a minimal and finite representation. This is taken care of by the following function, $\text{filter} : \mathcal{L}_4 \rightarrow \mathcal{L}_4$:

$$\begin{aligned} \text{filter}(\llbracket \psi \rrbracket \mapsto \ell, ls) &= \begin{cases} \text{filter}(ls) & \psi \equiv \text{false} \\ \llbracket \psi \rrbracket \mapsto \ell, \text{filter}(ls) & \psi \not\equiv \text{false} \end{cases} \\ \text{filter}(\text{nil}) &= \text{nil} \end{aligned}$$

The transfer function of $\mathcal{A}4$ thus has the following behavior on a statement, S , with feature constraint, ϕ :

$$\begin{array}{c} L_4 = (\llbracket \psi \rrbracket \mapsto \ell, \dots) \\ \downarrow \\ \boxed{\llbracket \phi \rrbracket: S} \\ \downarrow \\ L'_4 = \text{filter}(\llbracket \psi \wedge \phi \rrbracket \mapsto f_S(\ell), \llbracket \psi \wedge \neg \phi \rrbracket \mapsto \ell, \dots) \end{array}$$

Analysis: As always for the analysis, we simply combine the CFG, lattice, and transfer functions to achieve our shared simultaneous analysis. Figure 7(d) shows how this analysis will analyze our tiny program example from earlier. (For legibility, the last line of the figure is written with the expanded *sets of configurations* rather than with formula notation.) $\mathcal{A}3$ and $\mathcal{A}4$ compute the same information; $\mathcal{A}4$ just represents the same information more compactly using sharing.

$\mathcal{A}5$: Simultaneous Analysis with Sharing and Merging

In addition to configuration sharing as in $\mathcal{A}4$, it is also possible to *merge* sets of configurations that record the same information (i.e., sets of configurations that map to the same base lattice value). For instance, what in $\mathcal{A}4$ might be represented by $L_4 = (\llbracket \phi \rrbracket \mapsto +, \llbracket \psi \rrbracket \mapsto +)$ could instead be represented as $L_5 = (\llbracket \phi \vee \psi \rrbracket \mapsto +)$, essentially *merging* and thereby also sharing the sets of configurations, $\llbracket \phi \rrbracket$ and $\llbracket \psi \rrbracket$. Since each base lattice value in the co-domain of every $\mathcal{A}5$ lattice value is now unique, it seems more natural to reverse the lattice so as to represent the above information instead as $L_5 = (+ \mapsto \llbracket \phi \vee \psi \rrbracket)$ such that base lattice values are partially mapped to sets of configurations instead of the other way around.

Control-Flow Graph: The CFG of $\mathcal{A}5$ is the same as that of the previous feature-sensitive analyses ($\mathcal{A}2$, $\mathcal{A}3$, and $\mathcal{A}4$).

Lattice: As mentioned above, the lattice of $\mathcal{A}5$ now partially maps base lattice values to sets of configurations:

$$\mathcal{L}_5 = \mathcal{L} \hookrightarrow 2^{\llbracket \psi_{FM} \rrbracket}$$

Transfer Functions: The transfer functions of $\mathcal{A}5$ is easily defined as a transformation of representation, *merge*, from $\mathcal{L}_4$ to $\mathcal{L}_5$, which appropriately merges and disjoins formulae with equal base lattice values; i.e., $\text{merge} : \mathcal{L}_4 \rightarrow \mathcal{L}_5$, is defined as $L_5 = \text{merge}(L_4)$ where:

$$L_5(\ell) = \begin{cases} \bigvee_{L_4(\llbracket \phi \rrbracket) = \ell} \phi & \exists \phi. L_4(\llbracket \phi \rrbracket) = \ell \\ \text{undefined} & \text{otherwise} \end{cases}$$

A transfer function of $\mathcal{A}5$ for a statement, S , with feature constraint, ϕ , will thus have the following behavior:

$$\begin{array}{c} L_5 = (\ell \mapsto \llbracket \psi \rrbracket, \dots) \\ \downarrow \\ \boxed{\llbracket \phi \rrbracket: S} \\ \downarrow \\ L'_5 = \text{merge}(\text{filter}(\llbracket \psi \wedge \phi \rrbracket \mapsto f_S(\ell), \llbracket \psi \wedge \neg \phi \rrbracket \mapsto \ell, \dots)) \end{array}$$

Analysis: Again, we simply combine CFG, lattice, and transfer functions to achieve our reversed shared simultaneous analysis. Figure 7(e) illustrates how the analysis analyses our tiny increase-decrease method. (As in Figure 7(d), we write the last line of the figure as the expanded *sets of configurations* rather than using formula notation.) Of course, this particular example does not show off the merging capabilities. If we add statement, $x=0$, to the end of our tiny example method, we would get the analysis answer: $L_5 = (0 \mapsto \llbracket \psi_{FM} \rrbracket)$ instead of $L_4 = (\{\{A\}\} \mapsto 0, \{\{B\}\} \mapsto 0, \{\{A, B\}\} \mapsto 0)$. Note that there is a trade-off between the cost of computing the merging and the benefits of sharing. Whether or not this is worth while is likely to depend on a particular analysis (and SPL) and will have to be verified experimentally (cf. Section 7). In general, it would presumably work better on analyses with small lattices where lots of information could potentially be shared.

Other Analysis Approaches

A couple of variations of the feature-sensitive analyses are possible. One could retain the instrumented CFG calculated in $\mathcal{A}2$ and $\mathcal{A}3$, but then *specialize* [16] the CFG prior to analysis for every configuration by resolving all conditional statements relative to the current configuration. This approach would be a variation of $\mathcal{A}2$ with a higher cost due to CFG specialization, but which in turn saves by making membership decisions only once per CFG node. Another approach would be to transform `ifdefs` into normal `ifs` and turn feature names into static booleans [6, 29] which could then be resolved by techniques such as *partial evaluation* [16] prior to analysis. We do not explore this idea in the paper, but rather focus on the different ways of automatically transforming a feature-oblivious analysis into a feature-sensitive one, while staying within the framework of dataflow analysis.

6. Analysis of the Analyses

We will now analyze and compare the feature-sensitive analyses. We begin with a proof that all analyses $\mathcal{A}1$ to $\mathcal{A}5$ compute the same information. Then, we will outline the tasks performed by each of the analyses. Finally, we will reason about and compare the asymptotic time and space complexity of the feature-sensitive analyses.

Equivalence of the Analyses

Lemma 1: The analyses $\mathcal{A}1$, $\mathcal{A}2$, $\mathcal{A}3$, $\mathcal{A}4$, and $\mathcal{A}5$ all compute the exact same information (same analysis result).

Proof: It is sufficient to show that for each configuration and each statement, all transfer functions of $\mathcal{A}1$, $\mathcal{A}2$, $\mathcal{A}3$, $\mathcal{A}4$, and $\mathcal{A}5$ (cf. Figure 8) compute the exact same information (only represented in different ways by the five analyses). Then, the rest follows inductively by transitivity on the entire control-flow graph. Let configuration c and statement S

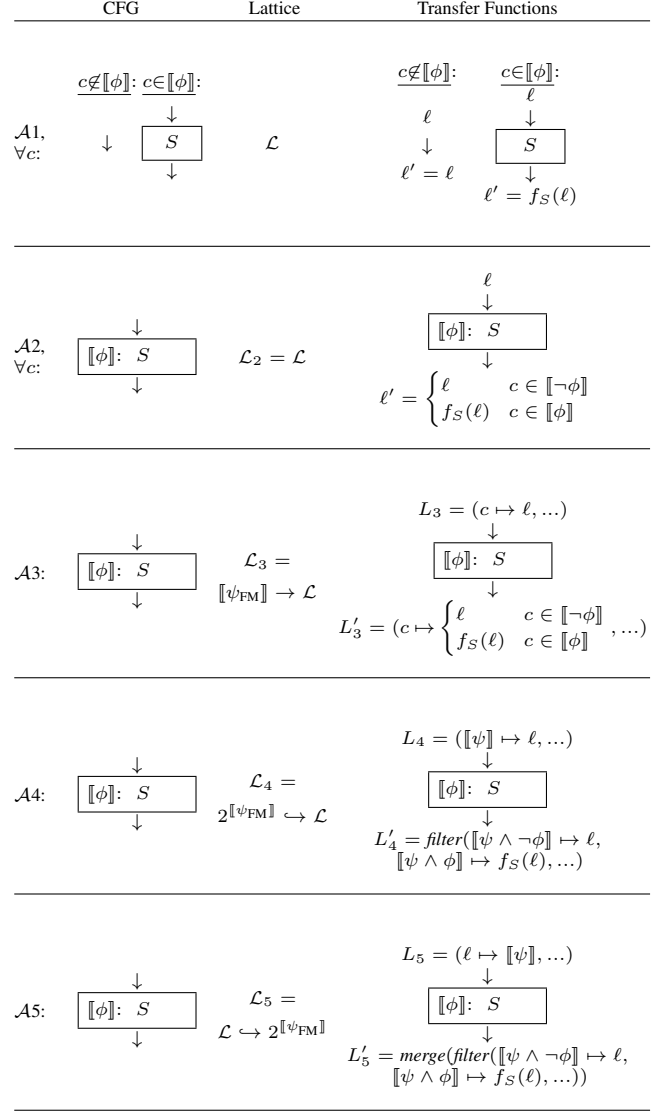


Figure 8. Summary of dataflow analyses for SPLs.

with feature constraint ϕ be given. We assume the input to the transfer function to associate base lattice value ℓ with the configuration c . We will now show that the resulting output of all of the transfer functions for $\mathcal{A}1$ to $\mathcal{A}5$ associate the same (new) output base lattice value, ℓ' , for our configuration c . Now, we have two cases, depending on whether or not $c \in \llbracket \phi \rrbracket$.

Case $c \in \llbracket \phi \rrbracket$:

- $\mathcal{A}1$: Since $c \in \llbracket \phi \rrbracket$, we get that: $\ell' = f_S(\ell)$.
- $\mathcal{A}2$: Here, we have that $\ell' = \begin{cases} \ell & c \in \llbracket \neg \phi \rrbracket \\ f_S(\ell) & c \in \llbracket \phi \rrbracket \end{cases}$ which means that $\ell' = f_S(\ell)$, as required, since $c \in \llbracket \phi \rrbracket$.

- $\mathcal{A3}$: By assumption, the input to the transfer function gives us that $L_3(c) = \ell$. Then, the output of the transfer function is $L'_3 = (c \mapsto \begin{cases} \ell & c \in \llbracket \neg\phi \rrbracket \\ f_S(\ell) & c \in \llbracket \phi \rrbracket \end{cases}, \dots)$ which means that the information computed for configuration c is $\ell' = L'_3(c) = f_S(\ell)$, as required, since $c \in \llbracket \phi \rrbracket$.
- $\mathcal{A4}$: Here, the input to the transfer function is assumed to associate $L_4(\llbracket \psi \rrbracket) = \ell$, for some ψ where $c \in \llbracket \psi \rrbracket$. The output of the transfer function is then $L'_4 = \text{filter}(\llbracket \psi \wedge \phi \rrbracket \mapsto f_S(\ell), \llbracket \psi \wedge \neg\phi \rrbracket \mapsto \ell, \dots)$. Since $c \in \llbracket \psi \rrbracket$ and $c \in \llbracket \phi \rrbracket$, we conclude that $c \in \llbracket \psi \wedge \phi \rrbracket$ which in turn means that $\psi \wedge \phi \neq \text{false}$ and hence that the information computed is $\ell' = L'_4(\llbracket \psi \wedge \phi \rrbracket) = f_S(\ell)$. This establishes that $c \in \llbracket \psi \wedge \phi \rrbracket$ is mapped to $\ell' = f_S(\ell)$, as required.
- $\mathcal{A5}$: Here, the input to the transfer function is assumed to associate $L_5(\ell) = \llbracket \psi \rrbracket$, for some ψ where $c \in \llbracket \psi \rrbracket$. In case $\mathcal{A4}$ above, we deduced that $L'_4(\llbracket \psi \wedge \phi \rrbracket) = f_S(\ell)$ for $L'_4 = \text{filter}(\llbracket \psi \wedge \phi \rrbracket \mapsto f_S(\ell), \llbracket \psi \wedge \neg\phi \rrbracket \mapsto \ell, \dots)$ and that $c \in \llbracket \psi \wedge \phi \rrbracket$. This then means that for $L'_5 = \text{merge}(L'_4)$ we have that $L'_5(f_S(\ell)) \supseteq \llbracket \psi \wedge \phi \rrbracket \ni c$. Hence, the information computed for configuration c is $\ell' = f_S(\ell)$, as required.

Case $c \notin \llbracket \phi \rrbracket$:

- $\mathcal{A1}$: Since $c \notin \llbracket \phi \rrbracket$, we get that: $\ell' = \ell$.
- $\mathcal{A2}$: Since $c \notin \llbracket \phi \rrbracket$, we deduce that $c \in \llbracket \neg\phi \rrbracket$. Now, by definition, $\ell' = \begin{cases} \ell & c \in \llbracket \neg\phi \rrbracket \\ f_S(\ell) & c \in \llbracket \phi \rrbracket \end{cases}$, we obtain that $\ell' = \ell$, as required.
- $\mathcal{A3}$: By assumption, the input to the transfer function gives us that $L_3(c) = \ell$. Then, the output of the transfer function is $L'_3 = (c \mapsto \begin{cases} \ell & c \in \llbracket \neg\phi \rrbracket \\ f_S(\ell) & c \in \llbracket \phi \rrbracket \end{cases}, \dots)$ which means that the information computed for configuration c is $\ell' = L'_3(c) = \ell$, as required, since $c \in \llbracket \neg\phi \rrbracket$.
- $\mathcal{A4}$: Here, the input to the transfer function is assumed to associate $L_4(\llbracket \psi \rrbracket) = \ell$, for some ψ where $c \in \llbracket \psi \rrbracket$. The output of the transfer function is $L'_4 = \text{filter}(\llbracket \psi \wedge \phi \rrbracket \mapsto f_S(\ell), \llbracket \psi \wedge \neg\phi \rrbracket \mapsto \ell, \dots)$. However, since $c \notin \llbracket \phi \rrbracket$, we conclude that $c \in \llbracket \neg\phi \rrbracket$ and then that $c \in \llbracket \psi \wedge \neg\phi \rrbracket$ which in turn means that $\llbracket \psi \wedge \neg\phi \rrbracket \neq \text{false}$ and hence that $\ell' = L'_4(\llbracket \psi \wedge \neg\phi \rrbracket) = \ell$. Thus, the information computed for $c \in \llbracket \psi \wedge \neg\phi \rrbracket$ is $\ell' = \ell$, as required.
- $\mathcal{A5}$: Here, the input to the transfer function is assumed to associate $L_5(\ell) = \llbracket \psi \rrbracket$, for some ψ where $c \in \llbracket \psi \rrbracket$. In case $\mathcal{A4}$ above, we deduced that $L'_4(\llbracket \psi \wedge \neg\phi \rrbracket) = \ell$ for $L'_4 = \text{filter}(\llbracket \psi \wedge \phi \rrbracket \mapsto f_S(\ell), \llbracket \psi \wedge \neg\phi \rrbracket \mapsto \ell, \dots)$ and that $c \in \llbracket \psi \wedge \neg\phi \rrbracket$. This then means that for $L'_5 = \text{merge}(L'_4)$ we have that $L'_5(\ell) \supseteq \llbracket \psi \wedge \neg\phi \rrbracket \ni c$ which means the information computed for configuration c is $\ell' = \ell$, as required.

$$\begin{aligned}
\text{tasks}(\mathcal{A1}) &= |\psi_{\text{FM}}| \cdot \text{compile} + |\psi_{\text{FM}}| \cdot \text{analyze}_{\mathcal{A1}} \\
\text{tasks}(\mathcal{A2}) &= \text{compile} + \text{instrument} + |\psi_{\text{FM}}| \cdot \text{analyze}_{\mathcal{A2}} \\
\text{tasks}(\mathcal{A3}) &= \text{compile} + \text{instrument} + \text{analyze}_{\mathcal{A3}} \\
\text{tasks}(\mathcal{A4}) &= \text{compile} + \text{instrument} + \text{analyze}_{\mathcal{A4}} \\
\text{tasks}(\mathcal{A5}) &= \text{compile} + \text{instrument} + \text{analyze}_{\mathcal{A5}}
\end{aligned}$$

Figure 9. Overall tasks performed by each of the analyses.

□

Analysis Tasks

Figure 9 considers the overall tasks performed for each SPL method analyzed in each of the analyses. Apart from $\mathcal{A3}$ to $\mathcal{A5}$, all analyses differ substantially in the number of times each of the tasks are performed. Not surprisingly, $\mathcal{A1}$ needs to do a lot of (brute force) compilation. The rest require only one compilation, but pay the price of instrumentation to annotate the CFG with feature constraints. However, this is cheap in practice. $\mathcal{A2}$ performs the analysis (i.e., the fixed-point computation) for every valid configuration whereas $\mathcal{A3}$ to $\mathcal{A5}$ only do this once. We return to these considerations, in practice, when we discuss our experimental results (cf. Section 7).

Asymptotic Time Complexity

The asymptotic time complexity of the $\mathcal{A2}$ analysis is:

$$\text{TIME}(\mathcal{A2}) = \mathcal{O}(|\psi_{\text{FM}}| \cdot |\mathcal{G}| \cdot T_2 \cdot h(\mathcal{L}_2))$$

where $|\mathcal{G}|$ is the size of the control-flow graph (which for the intraprocedural analysis is linear in the number of statements in the method analyzed, ignoring exceptions); T_2 is the execution time of a transfer function on the $\mathcal{L}_2$ lattice; and $h(\mathcal{L}_2)$ is the height of the $\mathcal{L}_2$ lattice. In total, we need to analyze $|\psi_{\text{FM}}|$ method variants. For each of these, we execute $\mathcal{O}(|\mathcal{G}|)$ different transfer functions, each of which takes execution time, T_2 , and can be executed a worst-case maximum of $h(\mathcal{L}_2)$ number of times.

Analogously, we can quantify the asymptotic time complexity of $\mathcal{A3}$:

$$\text{TIME}(\mathcal{A3}) = \mathcal{O}(|\mathcal{G}| \cdot T_3 \cdot h(\mathcal{L}_3))$$

which is similar to $\mathcal{A2}$, except that we do not need to analyze $|\psi_{\text{FM}}|$ times and that the numbers are parameterized by the $\mathcal{A3}$ lattice and transfer functions. For the height of the lattice $\mathcal{L}_3$, we have:

$$\begin{aligned}
h(\mathcal{L}_3) &= h(\llbracket \psi_{\text{FM}} \rrbracket \rightarrow \mathcal{L}_2) = h(\mathcal{L}_2^{|\psi_{\text{FM}}|}) \\
&= \sum_{c \in \llbracket \psi_{\text{FM}} \rrbracket} h(\mathcal{L}_2) = |\psi_{\text{FM}}| \cdot h(\mathcal{L}_2)
\end{aligned}$$

Note, however, that this is a purely theoretically worst case that does not naturally arise in practice because of the point-wise nature of $\mathcal{A3}$. Since all configurations are independent,

the penalty for $\mathcal{A}3$ will not be the *sum*, but rather only the *maximum* number of fixed-point iterations of $\mathcal{A}2$. In practice, we have not observed any significant cost on behalf of $\mathcal{A}3$ from this effect, as we will see in Section 7. The remaining speed factor between $\mathcal{A}2$ and $\mathcal{A}3$ thus boils down to:

$$\mathcal{A}2 : \mathcal{A}3 = |\psi_{\text{FM}}| \cdot T_2 : T_3$$

In theory, we would not expect any difference in the speed of the two analyses; $\mathcal{A}2$ makes a sequence of n analyses and $\mathcal{A}3$ makes one analysis in which each step costs n . However, as we will see in Section 7, $\mathcal{A}3$ has better cache performance than $\mathcal{A}2$, since statement nodes only have to be retrieved and evaluated once per transfer function in $\mathcal{A}3$, instead of once per configuration as in $\mathcal{A}2$. Apart from data, also the fixed-point iteration code only runs once instead of once per configuration.

The speed of the analyses $\mathcal{A}4$ and $\mathcal{A}5$ depends on the sharing potential which is intimately dependent on a particular SPL. In Section 7, we will see how these two analyses perform on our benchmarks.

Asymptotic SPACE Complexity

In terms of memory consumption, the asymptotic space complexity of the analyses is simply proportional to the amount of data occupied by the lattice values. For the analyses $\mathcal{A}2$ and $\mathcal{A}3$ these figure are:

$$\begin{aligned} \text{SPACE}(\mathcal{A}2) &= \mathcal{O}(|\mathcal{G}| \cdot \log(|\mathcal{L}_2|)) \\ \text{SPACE}(\mathcal{A}3) &= \mathcal{O}(|\mathcal{G}| \cdot \log(|\mathcal{L}_3|)) \end{aligned}$$

Comparing the two, we can derive that:

$$\begin{aligned} \log(|\mathcal{L}_3|) &= \log(|[\psi_{\text{FM}}] \rightarrow \mathcal{L}_2|) = \log(|\mathcal{L}_2|^{|\psi_{\text{FM}}|}) \\ &= \log(|\mathcal{L}_2|^{|\psi_{\text{FM}}|}) = |\psi_{\text{FM}}| \cdot \log(|\mathcal{L}_2|) \end{aligned}$$

which thus means that the difference is:

$$\text{SPACE}(\mathcal{A}3) = |\psi_{\text{FM}}| \cdot \text{SPACE}(\mathcal{A}2)$$

This relationship is also evident when comparing Figures 7(b) and 7(c). Although $\mathcal{A}3$ requires $n = |\psi_{\text{FM}}|$ times more memory to run, it is always possible to cut the $\mathcal{A}3$ lattice into k slices of n/k columns (i.e., analyze n/k number of configurations at a time). This provides a way to in some sense combine the time and space characteristics of $\mathcal{A}2$ with $\mathcal{A}3$ to $\mathcal{A}5$.

As for performance, the memory consumption of the analyses $\mathcal{A}4$ and $\mathcal{A}5$ depends on the sharing potential which will vary from SPL to SPL. Again, we will see how they fare in practice in Section 7.

7. Evaluation

We first present our study settings and then present our results in terms of total analysis time, performance, and memory consumption.

Benchmark	LOC	$ \mathbb{F} $	$ 2^{\mathbb{F}_{\text{local}}} $	avg. $ 2^{\mathbb{F}_{\text{local}}} $	#methods
Graph PL	1,350	18	$2^9 = 512$	7.44	135 (964)
MobileMedia08	5,700	14	$2^7 = 128$	2.88	285 (821)
Lampiro	45,000	11	$2^2 = 4$	1.01	1,949 (1,980)
BerkeleyDB	84,000	42	$2^8 = 256$	2.00	3,605 (7,446)

Figure 11. Size metrics for our four SPL benchmarks.

7.1 Study settings

To validate the ideas, we have implemented and evaluated the performance and memory characteristics of an intraprocedural version of the ubiquitous *reaching definitions* dataflow analysis (summarized in Figure 10) which is implemented using Soot’s interprocedural dataflow analysis framework for analyzing Java programs [34].

We have subsequently lifted them into the four feature-sensitive analyses for SPLs ($\mathcal{A}2$ to $\mathcal{A}5$). The sharing in the $\mathcal{A}4$ and $\mathcal{A}5$ analyses is represented by *bit vectors* from the Colt high performance open source Java libraries [1]. Since we are using intraprocedural analyses which analyze one method at a time, we can use the *local* set of features of each method, $\mathbb{F}_{\text{local}}$, instead of $\mathbb{F}$ which significantly reduces the size of the lattices we work with (recall that the number of configurations is exponential in the number of features).

We have chosen four qualitatively different SPL benchmarks, summarized in Figure 11. Graph PL (aka., GPL) is a small product line with a bit more than a thousand lines of code with intensive feature usage [20] for desktop applications. MobileMedia08 is a slightly larger product line with moderate feature usage [13] for mobile applications for dealing with multi-media. Lampiro is a moderately sized product line with low feature usage for instant-messaging clients [18]. Last but not least, BerkeleyDB is a product line with 84K lines of code for databases [20] of moderate feature usage. The table presented in Figure 11 summarizes: LOC, is the number of lines of code; $|\mathbb{F}|$, is the number of features in the SPL; $|2^{\mathbb{F}_{\text{local}}}|$, is the maximum number of configurations of any one method in the SPL; avg. $|2^{\mathbb{F}_{\text{local}}}|$ is the average number of configurations in a method and #methods, is the number of methods (with the total number of distinct methods, including variations of the same method, in parentheses). Methods completely encompassed by *ifdefs* are also counted.

The histograms in Figure 12 illustrate the distribution of the number of configurations per method for each of the SPLs. MobileMedia08 (depicted in Figure 12(b)), for instance, has 157 methods without features, 78 methods with one feature (i.e., $2^1 * 78 = 156$ distinct methods), 37 methods with two features (i.e., $2^2 * 37 = 148$ distinct methods) etc, and one method with seven features (i.e., $2^7 * 1 = 128$ distinct methods). The area shown in the histograms is thus directly proportional to the number of distinct methods to be analyzed.

The leftmost white bar represents methods without feature usage and thus trivially only one configuration. On such

Dataflow Analysis	Control-Flow Graph	Lattice	Transfer Functions:		Initialization
			$f_{x=E}(\ell)$	$f_{\text{int } x}(\ell)$	
Reaching Definitions	$\mathcal{G}_{\text{intraprocedural forward-flow}}$	$(2^{\text{ASSIGN}}, \subseteq)$	$(\ell \downarrow x) \cup \{x=E\}$	$\ell \downarrow x$	$\emptyset$

Figure 10. Specification of the *Reaching Definitions* analysis (where we write “ $\ell \downarrow x$ ” as a short-hand for “ $\ell \setminus \{x=E \mid \forall E\}$ ”).

methods, we expect the simultaneous analyses $\mathcal{A3}$ - $\mathcal{A5}$ to suffer an overhead penalty compared to $\mathcal{A2}$ from having to represent one base lattice value as either: in $\mathcal{A3}$, as a *list of one value*; or in $\mathcal{A4}$ and $\mathcal{A5}$, as a *set of one value*, trivially unsharable. We have thus plotted those bars in a different color (white) to emphasize the difference while the rest of the method bars are plotted in gray.

As can be seen, the four benchmark SPLs have qualitatively different feature usage profiles. The histogram of Graph PL is dominated by one method, `Vertex.display()`, with $2^9 = 512$ configurations. In *MobileMedia08*, there is one method, `MediaListScreen.initMenu()`, with $2^7 = 128$ configurations and one, `handleCommand()` in the class `MediaController`, with $2^6 = 64$. *Lampiro* only has one method, `RegisterScreen.xmppLogin()`, with $2^2 = 4$ configurations and 24 methods with $2^1 = 2$ configurations. In *BerkeleyDB*, the most configurations, $2^7 = 128$, is achieved by three methods³.

Our analyses currently interface with CIDE (Colored IDE) [20] for retrieving the conditional compilation statements. CIDE is a tool that enables developers to annotate feature code using background colors rather than `ifdefs` directives, reducing code pollution and improving comprehensibility. Conceptually, CIDE uses a restricted form of `ifdefs` for which only conjunction of features is permitted.

Our analyses assume that each line never has two parts with different CIDE colorings (i.e., different formulae). This is a fair assumption as lines with multiple configurations could be accommodated by inserting appropriate line breaks.

We have executed the analyses on a 64-bit machine with a Intel® Core™ i7 920 CPU running at a 2.6 GHz frequency with 8 GB of memory and 8MB L2 cache on a Linux Ubuntu 2.6.32-30-generic operating system.

7.2 Results and Discussion of $\mathcal{A2}$ vs. $\mathcal{A3}$ vs. $\mathcal{A4}$ vs. $\mathcal{A5}$

We now present the results⁴ obtained from our empirical study. We first present and discuss our results pertaining to the sum of analyzing all configurations for each SPL. First, we look at *total time* (incl. compilation), then at the *analysis-time only* (excl. compilation). Hereafter, we will look at the data for individual methods which give us indications on which analyses are best under what conditions which we exploit to intraprocedurally combine the analyses. Finally, we

look at memory consumption characteristics of our analyses. All times given are the second lowest data point (analysis sum total) over ten runs which appears to be the most stable number across repeated sets of ten runs.

On Total Time: Figure 13 plots the total time (including compilation) of the reaching definitions analysis on each of our four benchmarks. For the feature-oblivious brute-force analysis, $\mathcal{A1}$, the total time is shown in black whereas the feature-sensitive analyses, $\mathcal{A2}$ - $\mathcal{A5}$ are plotted in dark gray, gray, light gray, and white, respectively. All times comprise the tasks outlined in Figure 9. For each method, the time computed for $\mathcal{A1}$ is the average of the presumably *slowest* configuration ($c = 2^{\mathbb{F}}$) and the presumably *fastest* configuration ($c = \emptyset$) times the number of configurations to be considered. We have to do this estimation because several configurations, including ones that are valid according to the feature model, generate code that does not compile. Also, since the CIDE API does not currently provide an efficient way of getting the color of a line, we omit the time of this calculation from the CFG instrumentation time.

We see that the feature-sensitive analyses, $\mathcal{A2}$ to $\mathcal{A5}$, are all faster than the brute-force approach, $\mathcal{A1}$, except on *Lampiro* where they are all fairly equal (disregarding $\mathcal{A5}$). The “gain factor” of the best feature-sensitive analyses on each SPL is: 14 for Graph PL; 5.3 for *MobileMedia08*; 3.6 for *BerkeleyDB*; and not surprisingly, 1.0 for *Lampiro*. This give an average speed-up factor of *six* when compared to $\mathcal{A1}$.

The reason for this drastic speed-up is *compilation overhead* in that $\mathcal{A1}$ has to compile each method in each configuration (see Figure 9). On the other hand, $\mathcal{A2}$ to $\mathcal{A5}$ only need to compile each method *once* to obtain the instrumented CFG on which the analyses operate, even if many analyses are performed. In the following, we will thus focus on the times of the analyses themselves; i.e., without compilation and instrumentation.

On Analysis-Time Only: Figure 14 plots speed of the feature-sensitive analyses $\mathcal{A2}$ to $\mathcal{A5}$ using the reaching definitions analysis on all configurations of all methods.

We observe that $\mathcal{A4}$ is the considerably faster on Graph PL. This SPL has one method, `Vertex.display()`, with $2^9 = 512$ configurations that accounts for most of the analysis time and has many equivalent configurations that are indistinguishable by `#ifdefs` (i.e., for which the same `#ifdef` blocks apply). The sharing can thereby avoid many redundant basic transfer function computations. Yet, in the other

³The earlier AOSD 2012 version of this paper erroneously depicted a method in *BerkeleyDB* with 8 features in its feature-distribution histogram.

⁴All results including equivalent proofs are available at: <http://twiki.cin.ufpe.br/twiki/bin/view/SPG/EmergentAndDFA>

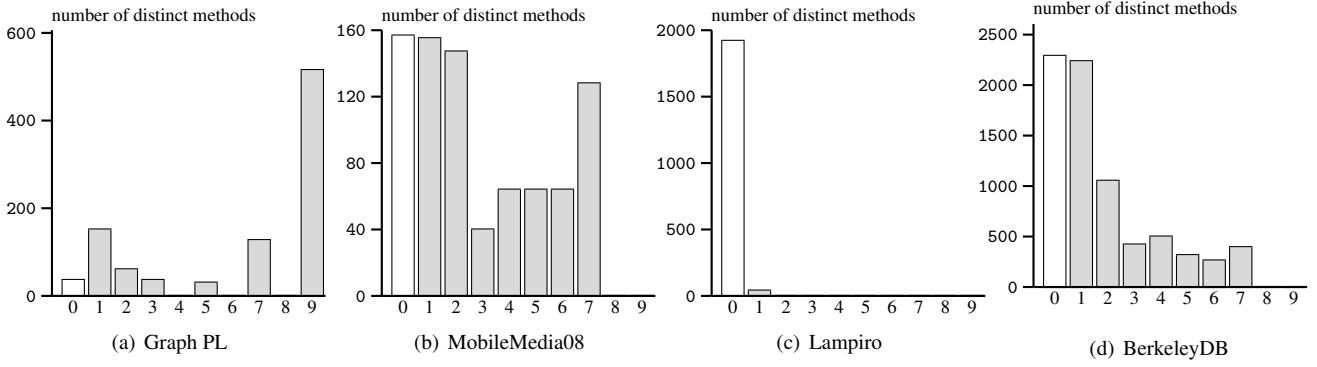


Figure 12. Histogram showing the distribution of number of features per method. The y-axis plots the number of distinct methods (e.g., Graph PL has *one* method with $2^9 = 512$ configurations which alone accounts for the entire rightmost bar).

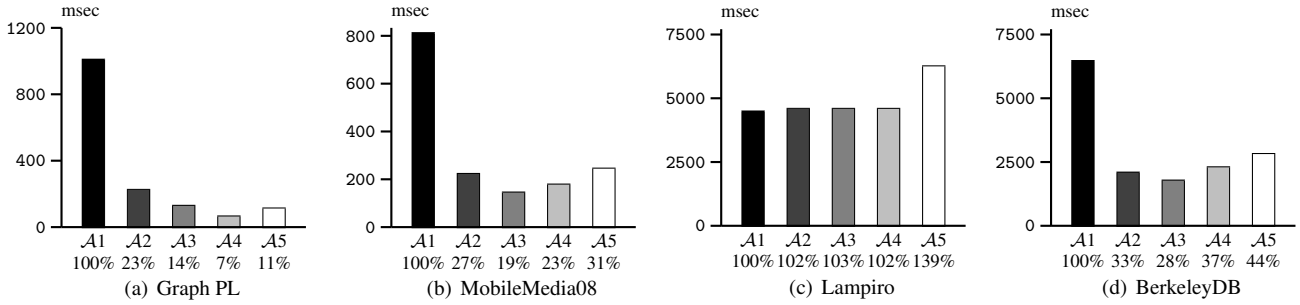


Figure 13. Total time for Reaching Definitions: .A1 (black) vs. .A2 (dark gray) vs. .A3 (gray) vs. .A4 (light gray) vs. .A5 (white).

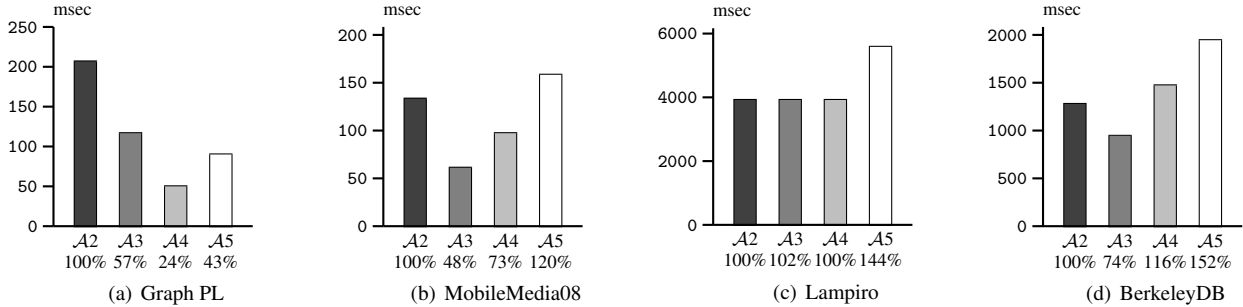


Figure 14. Analysis-only time for Reaching Definitions: .A2 (dark gray) vs. .A3 (gray) vs. .A4 (light gray) vs. .A5 (white).

cases (disregarding Lampiro) .A4 is slower than .A3 which is .A3 is the fastest on MobileMedia08 and BerkeleyDB. On Lampiro which has low feature usage, there does not appear to be any significant difference (apart from .A5 being slow). The merging of .A5 appears not to pay off in that it consistently loses to .A when looking at the sum of the analysis time for all methods.

On Analyzing with the Feature Model: Figure 15 plots the time of running reaching definitions on Graph PL and MobileMedia08 (excluding compilation) both *without* and *with* the feature model. Each bar is split into a left part where the feature model has been disabled (thereby analyzing *all* configurations) and a right part where the feature model has been enabled (analyzing *valid* configurations only). Predictably, and as hypothesized in an earlier ver-

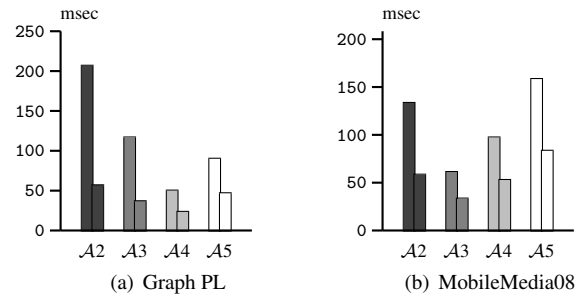


Figure 15. *Without* vs. *with* using the feature model (i.e., analyzing *all* configurations vs. *valid* configurations only).

sion of this paper [9], the *relative* speed among the analyses (i.e., the *pattern* of the histograms) does not change significantly when enabling the feature model so as to con-

sider only valid configurations. In absolute terms, however, the analysis time of course decreases; e.g., for $\mathcal{A}_2$, the time drops by a factor of close to four on Graph PL and a bit more than two on MobileMedia08. These benchmarks have proportionately many methods with intensive feature usage (cf. Figure 12) and thereby a large numbers of configurations that can decrease substantially when constrained by the feature model. For instance, Graph PL’s 512-configuration method has only 106 *valid* configurations. In MobileMedia08, the most feature intensive methods, `MediaListScreen.initMenu()` and `handleCommand()` in `MediaController`, respectively drop from 128 to 24 and 64 to 20 configurations.

In general, we expect SPLs with proportionately fewer methods with intensive feature usage (such as Lampiro and BerkeleyDB, cf. Figure 12), to be less affected when enabling the feature model.

On Caching and Data Locality: When looking at Figure 14, we see that $\mathcal{A}_3$ is consistently faster than $\mathcal{A}_2$. This is due to data locality and caching issues. In $\mathcal{A}_3$, base lattice values are represented as a list structure and processed in sequence on a node of the CFG. In terms of caching, this is better strategy than $\mathcal{A}_2$ which scatter computations and values across independent configuration runs; especially as the number of configurations increases.

The first data column of Figure 16, plots the ratio of cache misses between $\mathcal{A}_2$ and $\mathcal{A}_3$ under the “normal condition” when the L2 cache is enabled. As expected, $\mathcal{A}_2$ incurs quite a lot more cache misses than $\mathcal{A}_3$, making the former comparatively slower since data has to be re-fetched on every configuration. To further substantiate this caching hypothesis, we simulated L2 cache disabling, by instrumenting *both* the $\mathcal{A}_2$ and $\mathcal{A}_3$ with instructions to artificially fill up the L2 cache (by traversing an 8MB bogus array prior to transfer function execution). The second data column, *L2 cache disabled*, reveals that this indeed hurts $\mathcal{A}_2$ more than $\mathcal{A}_3$. We take this as evidence that $\mathcal{A}_3$ is better at exploiting the L2 cache than $\mathcal{A}_2$ under the “normal condition”.

Benchmark	$\mathcal{A}_2:\mathcal{A}_3$ (L2 cache enabled)	$\mathcal{A}_2:\mathcal{A}_3$ (L2 cache disabled)
Graph PL	+4%	+13%
MobileMedia08	+36%	+45%
Lampiro	+12%	+18%
BerkeleyDB	+21%	+29%

Figure 16. Cache misses in $\mathcal{A}_2$ vs. $\mathcal{A}_3$ under two different L2 caching schemes (*cache enabled* and *cache disabled*).

Recall that $\mathcal{A}_3$ in principle has to do as many fixed-point iterations as are needed for the slowest converging configuration, unnecessarily reiterating already converged configurations. Our data, however, indicates that this is not a problem in practice. For example, in the reaching definitions analysis, $\mathcal{A}_3$ only executes as little as 0.22% more unlifted transfer functions than $\mathcal{A}_2$ on BerkeleyDB; only 0.05%

more on Graph PL; and virtually 0% more on Lampiro and MobileMedia08.

On Combining the Analyses: In *intraprocedural* analysis, we analyze one method at a time which means that we can analyze methods using different analyses. We will now look at how to *combine* our analyses by selecting different analyses for different methods so as to achieve a more effective analysis strategy.

So far, the discussion has focused on the sum of the time for analyzing *all* methods in each given SPL. By looking at the data for each method in isolation, we can see that the four feature sensitive approaches behave differently for different kinds of methods. A number of factors, including *number of configurations*, *number of statements*, and (particularly relevant to reaching definitions) the *number of assignments*, have an effect on the performance of our approaches.

Knowledge on which of our analyses are good under what conditions may allow us to use heuristics to combine the analyses to obtain a combined analysis which is even faster than any of $\mathcal{A}_2$ to $\mathcal{A}_5$, individually. In the following, we will investigate one factor, namely the number of configurations as a predictor of which analysis is fastest.

When we look at which analyses are best on which methods, interesting information emerges. But first, let us construct a hypothetically ideal analysis strategy, $\mathcal{A}_{\text{IDEAL}}$, which is unrealistically allowed to use an oracle to decide the fastest analysis on each method. This strategy will provide a bound on the best we can hope for with this combination approach, given the implementations of our feature-sensitive analyses.

We see that $\mathcal{A}_4$ is close to optimal on Graph PL (50 msec for $\mathcal{A}_4$ vs. 49 msec for $\mathcal{A}_{\text{IDEAL}}$). Also, $\mathcal{A}_3$ is not too far away from the ideal on MobileMedia08 (61 vs. 53 msec) and BerkeleyDB (0.91 vs. 0.80 sec).

Figure 17 plots how often (as a percentage) each of the analyses $\mathcal{A}_2$ to $\mathcal{A}_5$ is the fastest (i.e., win) on methods with a varying number of configurations (plotted along the x-axis). We clearly see tendencies. At the far left of the histogram, we see that $\mathcal{A}_2$ (in dark gray) almost always wins on configurations of size $2^0 = 1$ and most frequently wins on methods with $2^1 = 2$ configurations. At the far right, we see that $\mathcal{A}_4$ (in light gray) wins on the one 512-configuration method of Graph PL. $\mathcal{A}_3$ is generally the winner in the middle range, if we lump all methods with 4-128 configurations together as an uninterrupted interval (except, of course, for the 8 and 32 case where it loses a bit to $\mathcal{A}_4$).

This gives rise to the following strategy, $\mathcal{A}^*$, which takes a number of configurations, n , and then selects one of our analyses:

$$\mathcal{A}^*(n) = \begin{cases} \mathcal{A}_2 & n \leq 2 \\ \mathcal{A}_3 & 4 \leq n \leq 128 \\ \mathcal{A}_4 & 256 \leq n \end{cases}$$

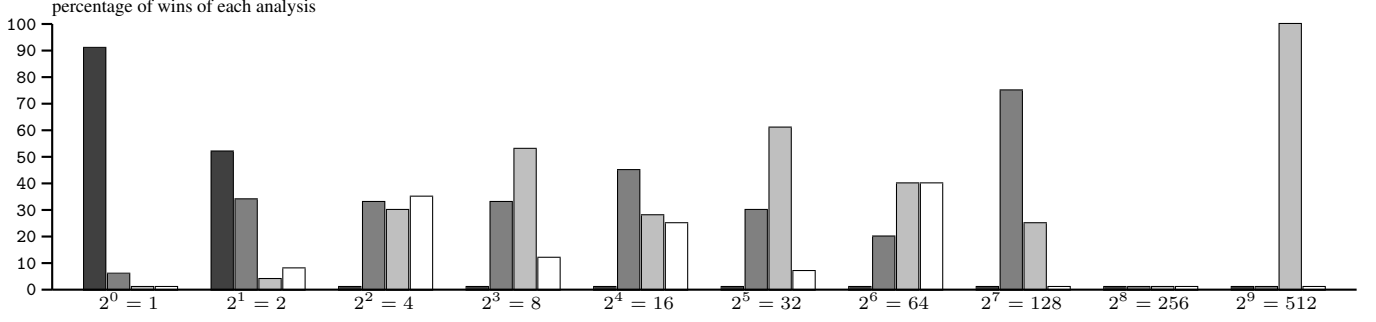


Figure 17. Histogram showing the percentage of wins of each analysis (along the y-axis) on methods (in all four SPLs) with a certain number of configurations (plotted along the x-axis). $\mathcal{A}2$ (dark gray) vs. $\mathcal{A}3$ (gray) vs. $\mathcal{A}4$ (light gray) vs. $\mathcal{A}5$ (white).

These choices also make sense if we remember that $\mathcal{A}2$ has less overhead on methods with a low number of configurations and that $\mathcal{A}4$ will have more values to share on methods with a higher number of configurations. However, this relationship is correlative, not causal.

When evaluated, this combined analysis strategy, $\mathcal{A}^*$, is consistently *slightly faster* than each of its constituent analyses on all four benchmarks. Interestingly, the speed up is less than one percentage point of $\mathcal{A}2$ time (cf. Figure 14(a-d)); for this reason, we have not plotted $\mathcal{A}^*$ in the histograms.

The gain factors for $\mathcal{A}^*$ compared to $\mathcal{A}1$ are thus: 14 on Graph PL, 5.3 on MobileMedia08, 1.0 on Lampiro, and 3.6 on BerkeleyDB (which is on average six times faster than $\mathcal{A}1$ over our four benchmarks).

Although $\mathcal{A}5$ wins on several occasions (especially in the higher range, cf. Figure 17), it is also considerably slower on many methods (as also evident in Figure 14(b-d)). For this reason, we will refrain from using it in our heuristic strategy. Of course, having the heuristic strategy depend on more factors to the heuristics, such as the number of statements or assignments in a method, is likely to improve analysis time even further.

Memory Consumption of Analyses (SPACE): Figure 18 lists the space consumption by the maximum memory consuming method (wrt. $\mathcal{A}3$). Our experimental data confirms that $\mathcal{A}3$ requires almost $|2^{\mathbb{I}_{\text{local}}}|$ times more memory than $\mathcal{A}2$ since it has to keep all configurations in memory during its fixed-point computation. This does, however, not appear to be a problem in practice for intraprocedural analysis as it only needs to keep data for one method in memory at a time. Indeed, this has not been a problem on any of our four benchmarks. The shared analysis $\mathcal{A}4$ may reduce space usage anywhere between a factor of one to 15, depending on the SPL.

8. Related Work

Data-Flow Analysis: The idea of making dataflow analysis sensitive to statements that may or may not be executed is related to *path-sensitive* dataflow analysis. Such analyses compute different analysis information along different execution paths aiming to improve precision by disregarding

spurious information from infeasible paths [7] or to optimize frequently executed paths [4]. Earlier, disabling infeasible dead statements has been exploited to improve the precision of constant propagation [35] by essentially running a dead-code analysis capable of tagging statements as executable or non-executable during constant propagation analysis.

Predicated dataflow analysis [26] introduced the idea of using propositional logic predicates over runtime values to derive so-called *optimistic* dataflow values guarded by predicates. Such analyses are capable of producing multiple analysis versions and keeping them distinct during analysis much like our simultaneous analyses. However, their predicates are over dynamic state rather than SPL feature constraints for which everything is statically decidable.

The novelty in our paper is the application of the dataflow analysis framework to the domain of SPLs giving rise to the concept of *feature-sensitive* analyses that take conditionally compiled code and feature models into consideration so as to analyze not one program, but an entire SPL family of programs.

Analysis of SPLs: In SPLs, there are features whose presence or absence do not influence the test outcomes, which makes many feature combinations unnecessary for a particular test. This idea of selecting only relevant features for a given test case was proposed in a recent work [15]. It uses dataflow analysis to recover a list of features that are reachable from a given test case. The unreachable features are discarded, decreasing the number of combinations to test. In contrast, we defined and demonstrated how to automatically make any conventional dataflow analysis able to analyze SPLs in a feature-sensitive way. Thus, our feature-sensitive idea might be used in such a work (testing). For example, it might further reduce the time spent figuring out which relevant feature combinations to test.

We recently proposed the concept of emergent interfaces [31]. These interfaces emerge on demand to give support for specific SPL maintenance and thus help developers understand and manage dependencies between features. Feature dependencies such as assigning a value to a variable used by another feature, have to be generated by feature-sensitive analyses. Thus, our present work may be used to

Benchmark	Max. memory consuming method	$ 2^{\mathcal{E}_{\text{local}}} $	$\mathcal{A}2$	$\mathcal{A}3$ ($\mathcal{A}2:\mathcal{A}3$)	$\mathcal{A}4$ ($\mathcal{A}3:\mathcal{A}4$)
Graph PL	Vertex.display()	$2^9 = 512$	37 KB	9.9 MB (1:281)	1.4 MB (7.2:1)
MobileMedia08	MediaController.handleCommand()	$2^6 = 64$	93 KB	4.8 MB (1:53)	2.5 MB (1.9:1)
Lampiro	InfTree.clinit()	$2^0 = 1$	12 MB	12 MB (1:1)	12 MB (1:1)
BerkeleyDB	DbRunAction.main()	$2^7 = 128$	212 KB	20 MB (1:96)	1.3 MB (15:1)

Figure 18. Maximum memory consumption of lattice information during analysis ($\mathcal{A}2$ vs. $\mathcal{A}3$ vs. $\mathcal{A}4$).

generate emergent interfaces to support SPL maintenance. Our analyses are more efficient than the brute force approach, which is important to improve the performance during the computation of emergent interfaces.

Recently and inspired on our work, Bodden proposed [8] a mechanism for encoding a dataflow analyses specified via the IFDS framework [30] in a graph representation using the more general IDE analysis framework [32], thereby essentially “lifting” an IFDS dataflow analysis. The idea, although supposedly implemented, has not yet been evaluated. Interestingly, all of our analysis approaches $\mathcal{A}1$ to $\mathcal{A}5$ apply equally to that work as well and insights from this paper could benefit in that work.

Lifting for SPLs: Researchers already lifted automated analysis and processing such as for parsing [21], model checking [10], monitoring [23], type checking [5], and verification [6, 29]. Kaestner et al. [21] provides a variability-aware parser which is capable of parsing code without pre-processing it. The parser also performs instrumentation as we do, but on tokens, instead of statements. When the parser reaches a token instrumented with feature A , it splits into branches. Then, one parser assumes that feature A is selected and another assumes that A is not. So, the former consumes the token and the latter skips it. To avoid parsing tokens repeatedly (like a parenthesis instrumented with no feature), the branches are joined. This approach is similar to our shared simultaneous analyses $\mathcal{A}4$ and $\mathcal{A}5$, where we lazily split and merge sets of configurations.

Classen et al. [10] shows that behavioral models offer little means to relate different products and their respective behavioral descriptions. To minimize this limitation, they present a transition system to describe the combined behavior of an entire SPL. Additionally, they provide a model checking technique supported by a tool capable of verifying properties for all the products of an SPL once. Like our work, they claim that checking all product combinations at once instead of each product separately is faster. Their model checking algorithm was on average 3.5 times faster than verifying products separately.

Safe composition: Safe composition (SC) relates to the safe generation and verification of properties for SPL assets providing guarantees that the product derivation process generates products with properties that are obeyed [5, 19]. Safe composition may help in finding problems like undeclared variables. We complement safe composition, since when using our feature-sensitive idea, we are able to catch

any errors expressible as a dataflow analysis (e.g., uninitialized variables and null pointers).

9. Conclusion

In this paper, we presented four approaches for taking any standard one-program dataflow analysis and automatically lifting it into a feature-sensitive analysis capable of analyzing all configurations of an SPL. To evaluate these approaches, we applied them the ubiquitous reaching definitions dataflow analysis and lifted it to produce four different feature-sensitive analyses. Experimental validation indicates that the feature-sensitive approaches are faster than the naive brute-force approach. From this we were able to derive an analysis strategy which uses the number of configurations in a method in order to heuristically select the best analysis for the method. This analysis strategy is faster than all of our lifted analyses on all benchmarks. On average over the benchmarks, this strategy is six times faster than the existing alternative ($\mathcal{A}1$).

On our biggest SPL which also takes the longest to analyze, BerkeleyDB, we are able to cut the total analysis time of all configurations from 6.3 to 1.8 seconds using our simultaneous feature-sensitive approach, $\mathcal{A}3$. On Graph PL, which has equivalent configurations that can be shared by $\mathcal{A}4$, we can cut the time from a bit more than a second to less than 0.08 seconds using this analysis strategy. On average over the benchmarks, our analysis strategy is six times faster than the existing alternative ($\mathcal{A}1$).

We also conclude that our four approaches have different performance and memory consumption characteristics. The simultaneous feature-sensitive analysis ($\mathcal{A}3$) is, in general, the fastest. On the other hand, in terms of memory consumption, $\mathcal{A}4$ performs better than $\mathcal{A}3$. Finally, lack of memory problems does not show up as a problem in practice for intraprocedural analysis on the benchmarks.

Acknowledgments

We would like to thank CNPq, FACEPE, and National Institute of Science and Technology for Software Engineering (INES), for partially supporting this work. Also, we thank SPG⁵ members for the fruitful discussions about this paper. We also thank Julia Lawall for the comments that helped to improve this paper.

⁵<http://www.cin.ufpe.br/spg>

References

- [1] The colt project: Open source libraries for high performance scientific and technical computing in java. URL <http://acs.lbl.gov/software/colt/>. CERN: European Organization for Nuclear Research.
- [2] B. Adams, W. De Meuter, H. Tromp, and A. E. Hassan. Can we refactor conditional compilation into aspects? In *Proceedings of the 8th ACM international conference on Aspect-oriented software development (AOSD'09)*, pages 243–254, Charlottesville, Virginia, USA, 2009. ACM.
- [3] S. B. Akers. Binary decision diagrams. *IEEE Transactions on Computers*, 27:509–516, June 1978.
- [4] G. Ammons and J. R. Larus. Improving data-flow analysis with path profiles. In *Programming Language Design and Implementation (PLDI'98)*, pages 72–84, Montreal, Canada, 1998.
- [5] S. Apel, C. Kästner, A. Grösslinger, and C. Lengauer. Type safety for feature-oriented product lines. *Automated Software Engineering*, 17:251–300, September 2010.
- [6] S. Apel, H. Speidel, P. Wendler, A. von Rhein, and D. Beyer. Detection of feature interactions using feature-aware verification. In *Proceedings of the 26th IEEE/ACM International Conference on Automated Software Engineering (ASE'11)*, Lawrence, USA, November 2011. IEEE Computer Society.
- [7] T. Ball and S. K. Rajamani. Bebop: a path-sensitive interprocedural dataflow engine. In *PASTE'01*, pages 97–103, Snowbird, Utah, USA, 2001.
- [8] E. Bodden. Position Paper: Static flow-sensitive & context-sensitive information-flow analysis for software product lines. In *ACM SIGPLAN Seventh Workshop on Programming Languages and Analysis for Security (PLAS 2012)*, jun 2012. URL <http://www.bodden.de/pubs/bodden12static.pdf>.
- [9] C. Brabrand, M. Ribeiro, T. Tolêdo, and P. Borba. Intraprocedural dataflow analysis for software product lines. In *AOSD*, pages 13–24, 2012.
- [10] A. Classen, P. Heymans, P.-Y. Schobbens, A. Legay, and J.-F. Raskin. Model checking lots of systems: efficient verification of temporal properties in software product lines. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering (ICSE '10)*, pages 335–344, Cape Town, South Africa, 2010. ACM.
- [11] P. Clements and L. Northrop. *Software Product Lines: Practices and Patterns*. Addison-Wesley, 2001.
- [12] M. D. Ernst, G. J. Badros, and D. Notkin. An empirical analysis of c preprocessor use. *IEEE Transactions on Software Engineering*, 28:1146–1170, December 2002.
- [13] E. Figueiredo, N. Cacho, C. Sant'Anna, M. Monteiro, U. Kulesza, A. Garcia, S. Soares, F. Ferrari, S. Khan, F. C. Filho, and F. Dantas. Evolving software product lines with aspects: an empirical study on design stability. In *Proceedings of the 30th International Conference on Software Engineering (ICSE'08)*, pages 261–270, Leipzig, Germany, 2008. ACM.
- [14] M. Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.
- [15] C. Hwan, P. Kim, D. Batory, and S. Khurshid. Reducing combinatorics in testing product lines. In *Proceedings of the 10th International Conference on Aspect-oriented Software Development (AOSD'11)*, pages 57–68, Porto de Galinhas, Brazil, 2011. ACM.
- [16] N. D. Jones, C. K. Gomard, and P. Sestoft. *Partial evaluation and automatic program generation*. Prentice-Hall, Inc., Upper Saddle River, NJ, USA, 1993.
- [17] K. C. Kang, S. G. Cohen, J. A. Hess, W. E. Novak, and A. S. Peterson. Feature-Oriented Domain Analysis (FODA) feasibility study. Technical report, Carnegie-Mellon University Software Engineering Institute, November 1990.
- [18] C. Kästner. *Virtual Separation of Concerns: Toward Preprocessors 2.0*. PhD thesis, University of Magdeburg, Germany, May 2010.
- [19] C. Kästner and S. Apel. Type-checking software product lines - a formal approach. In *Proceedings of the 23rd IEEE/ACM International Conference on Automated Software Engineering (ASE'08)*, pages 258–267, L'Aquila, Italy, 2008.
- [20] C. Kästner, S. Apel, and M. Kuhlemann. Granularity in software product lines. In *Proceedings of the 30th International Conference on Software Engineering (ICSE'08)*, pages 311–320, Leipzig, Germany, 2008. ACM.
- [21] C. Kästner, P. G. Giarrusso, T. Rendel, S. Erdweg, K. Ostermann, and T. Berger. Variability-aware parsing in the presence of lexical macros and conditional compilation. In *Proceedings of the ACM International Conference on Object-Oriented Programming Systems Languages and Applications (OOPSLA'11)*, pages 805–824, Portland, OR, USA, 2011. ACM.
- [22] G. A. Kildall. A unified approach to global program optimization. In *Proceedings of the 1st annual ACM symposium on Principles of programming languages (POPL'73)*, pages 194–206, Boston, Massachusetts, 1973. ACM.
- [23] C. H. P. Kim, E. Bodden, D. Batory, and S. Khurshid. Reducing configurations to monitor in a software product line. In *1st International Conference on Runtime Verification (RV)*, volume 6418 of *LNCS*, Malta, November 2010. Springer.
- [24] M. Krone and G. Snelling. On the inference of configuration structures from source code. In *Proceedings of the 16th International Conference on Software Engineering (ICSE'04)*, pages 49–57, Los Alamitos, CA, USA, 1994. IEEE Computer.
- [25] J. Liebig, S. Apel, C. Lengauer, C. Kästner, and M. Schulze. An analysis of the variability in forty preprocessor-based software product lines. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering (ICSE'10)*, pages 105–114, Cape Town, South Africa, 2010. ACM.
- [26] S. Moon, M. W. Hall, and B. R. Murphy. Predicated array data-flow analysis for run-time parallelization. In *Proceedings of the 12th International Conference on Supercomputing (ICS'98)*, pages 204–211, Melbourne, Australia, 1998. ACM.
- [27] F. Nielson, H. R. Nielson, and C. Hankin. *Principles of Program Analysis*. Springer-Verlag, Secaucus, USA, 1999.
- [28] K. Pohl, G. Böckle, and F. van der Linden. *Software Product Line Engineering: Foundations, Principles and Techniques*. Springer, 2005.

- [29] H. Post and C. Sinz. Configuration lifting: Verification meets software configuration. In *Proceedings of the 23rd IEEE/ACM International Conference on Automated Software Engineering (ASE'08)*, pages 347–350, L'Aquila, Italy, 2008. IEEE Computer Society.
- [30] T. Reps, S. Horwitz, and M. Sagiv. Precise interprocedural dataflow analysis via graph reachability. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, POPL '95, pages 49–61, New York, NY, USA, 1995. ACM. ISBN 0-89791-692-1.
- [31] M. Ribeiro, H. Pacheco, L. Teixeira, and P. Borba. Emergent feature modularization. In *Onward! 2010, affiliated with the 1st ACM SIGPLAN International Conference on Systems, Programming, Languages and Applications: Software for Humanity (SPLASH'10)*, pages 11–18, Reno, NV, USA, 2010.
- [32] M. Sagiv, T. Reps, and S. Horwitz. Precise interprocedural dataflow analysis with applications to constant propagation. In *Selected papers from the 6th international joint conference on Theory and practice of software development*, TAPSOFT '95, pages 131–170, Amsterdam, The Netherlands, The Netherlands, 1996. Elsevier Science Publishers B. V.
- [33] H. Spencer and G. Collyer. #ifdef considered harmful, or portability experience with C news. In *Proceedings of the Usenix Summer 1992 Technical Conference*, pages 185–198, Berkeley, CA, USA, June 1992. Usenix Association.
- [34] R. Vallée-Rai, P. Co, E. Gagnon, L. Hendren, P. Lam, and V. Sundaresan. Soot - a java bytecode optimization framework. In *Proceedings of the 1999 conference of the Centre for Advanced Studies on Collaborative research (CASCON'99)*, pages 13–. IBM Press, 1999.
- [35] M. N. Wegman and F. K. Zadeck. Constant propagation with conditional branches. *ACM Transactions on Programming Languages and Systems*, 13:181–210, April 1991.

List of Paper Extensions

Note to the reviewers: For your convenience, here is the list of qualitative improvements and extensions compared to the AOSD 2012 conference paper (quantitatively, the paper has been extended from 12 to 16.5 pages):

- **A5:** a new feature-sensitive analysis, $\mathcal{A}_5$, with merging;
- **Proofs:** a proof of equivalence of the analyses;
- **Evidence:** on which analyses are good under what conditions (correlation between #configurations and fastest analysis);
- **Combined analyses:** $\mathcal{A}^*$ which is better on all SPLs;
- **FM:** on analyzing with vs. without the feature model;
- **Examples:** three new motivating examples in Section 2;
- **Various:** more explanation, figures, and related work.